

实验8: DQN回顾

Part 1 理论回顾

- ▶ 为什么需要值函数近似?
- ▶ Q-learning 与 Bellman 最优方程
- ▶ DQN 的损失函数定义
- ▶ 为什么梯度计算变得棘手
- ▶ 关键技术 ① 目标网络
- ▶ 关键技术 ② 经验回放
- ▶ DQN 完整算法伪代码

Part 2 代码详解

- ▶ Atari Pong 环境与图像预处理
- ▶ TorchRL × TensorDict 框架
- ▶ 经验回放缓冲区的实现
- ▶ Q 网络 CNN 架构与维度计算
- ▶ ϵ -greedy 策略代码细节
- ▶ Double DQN 训练循环
- ▶ gather / smooth_l1_loss 详解



第一部分 · 理论回顾

为什么需要值函数近似?

表格方法 (Tabular)

state	a_1	a_2	a_3
s_1	0.2	-0.1	0.4
s_2	0.1	0.3	0.0
s_3	0.5	0.2	-0.3
$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_n	?	?	?

✗ 当 $|S|, |A|$ 极大或连续时崩溃

- ▶ 存储: Atari 一帧就有 $256^{(210 \times 160 \times 3)}$ 种状态
- ▶ 泛化: 相似状态学不到共享经验
- ▶ 采样: 必须访问到每个 (s, a) 才能学习

函数近似 (Function Approximation)

$$\hat{V}(s, w) \approx v\pi(s)$$

用一组参数 $w \in \mathbb{R}^m$ (远少于 $|S|$) 参数化一个函数。
在 DQN 里, w 就是一张神经网络的所有权重。

✓ 解决两大问题

- ▶ 存储: w 维度 $\ll |S|$
- ▶ 泛化: 更新一处 $\rightarrow$ 邻近状态同步改变
- ▶ 代价: 值不再精确表示, 故称"近似"

表格 vs 函数：两种关键操作的差异

	表格方法	函数方法
读取值	直接 $Q[s,a]$ 查表	前向传播 $\hat{v} = \phi^T(s)w$ 或 $NN(s,w)$
更新值	直接 $Q[s,a] \leftarrow new$	通过更新 w 间接改变；牵一发而动全身
存储	$O(S \cdot A)$	$O(\dim w) \ll O(S \cdot A)$
泛化	无：(s,a) 互相独立	强：相似 s 共享参数
可扩展性	状态空间大就崩溃	可处理像素、高维连续状态

► 函数方法用「更新参数 w 」取代「修改表格条目」—— 这就是后面所有 TD/Sarsa/Q-learning 推导的核心切换点。

回到 Q-learning: 它要学什么?

动作值函数 $Q^*(s, a)$

$$Q^*(s, a) = E[R + \gamma \max_{a'} Q^*(s', a')]$$

这就是著名的 Bellman 最优方程

表格 Q-learning 更新规则

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- ▶ **TD target** $y = r + \gamma \max_{a'} Q(s', a')$
- ▶ **TD error** $\delta = y - Q(s, a)$

三个核心思想

自举 (Bootstrapping)

用当前估计 $Q(s', a')$ 去构造目标 y , 不必等到回合结束。

时序差分 (TD)

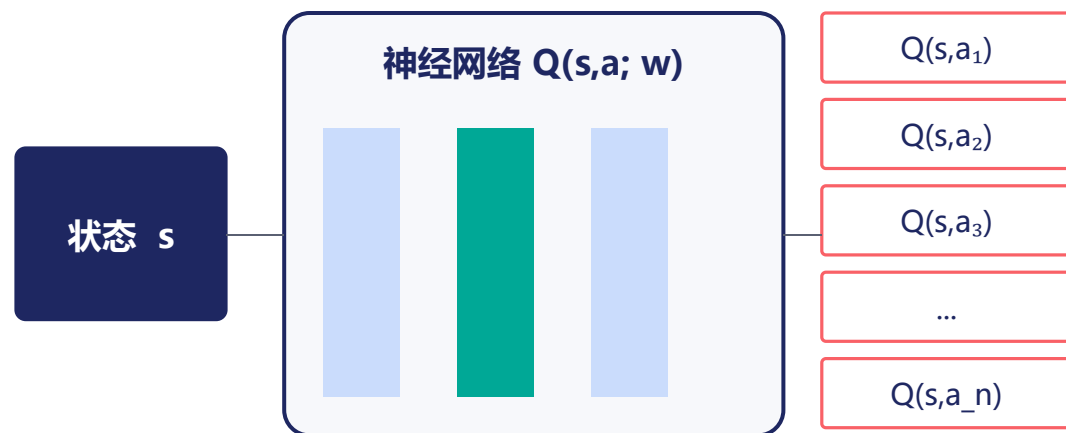
目标 y 与当前 $Q(s, a)$ 的差, 就是要消除的 TD error。

Off-policy

策略 $\max_{a'}$ 是贪心的, 但采样 a 可以用 ϵ -greedy —— 两者不需要一致。

把 Q-table 换成神经网络: DQN 的起点

Q-network 的接口形式



关键观察

一次前向 $\rightarrow$ n 个动作值

输入只看 s , 一次前向输出整个动作维度的 Q 值; 选动作时只需 argmax 。

同一份 w 共享所有 (s, a)

状态相似 $\Rightarrow$ 输出相似。学一个 (s,a) 等于同时改变邻居们的估计 —— 这正是泛化。

像素当输入: CNN 自然登场

Atari 中 s 是 $84 \times 84 \times 4$ 灰度帧堆叠。
卷积层提取空间特征, 全连接层映射到动作维度 — 即 Lab09 中的 DQN 类。

DQN 的损失函数（目标函数）

$$L(w) = E[(R + \gamma \max_{a'} Q(s', a'; w) - Q(s, a; w))^2]$$

TD 目标 y

$y = r + \gamma \cdot \max_{a'} Q(s', a'; w)$
「我们希望 Q 应该等于的值」

当前估计

$Q(s, a; w)$
「网络当前给出的值」

误差平方

$(y - Q)^2 \rightarrow \text{最小化}$
「让估计逐步向 Bellman 目标对齐」

↪ Lab 09 里这正对应：

```
target    = rew_b + gamma * (1.0 - done_b) * q_next    # 这就是 y
q_values  = q(obs_b).gather(1, act_b_ind.unsqueeze(-1)).squeeze(1)
loss      = F.mse_loss(q_values, target)               # 最小化 (y - Q)2
```


棘手的梯度：目标里也藏着 w

目标 y 与估计 Q 都依赖同一组参数 w

$$y = r + \gamma \max_{a'} Q(s', a'; w) \quad \leftarrow w \text{ 在这里}$$

- ▶ 如果直接对 w 求梯度，会同时影响「目标」和「估计」两侧
- ▶ 训练会变得极不稳定 —— 一步更新就让目标也跟着移动，像追着自己的影子
- ▶ 经典称呼：deadly triad (off-policy + function approximation + bootstrapping)

💡 解决思路

在计算梯度时，假设目标 y 里的 w 是「冻结的常量」。

用一个延迟更新的参数副本 w^T 来产生目标 y —— 这就引出了「目标网络」。

关键技术 ① — 目标网络 (Target Network)

两套权重: 主网络 q + 目标网络 q_{target}

主网络 $q(s, a; w)$

每一步都通过
反向传播更新
参数 w

目标网络 $q(s, a; w^T)$

只用于计算 y
参数固定 C 步后
才整体复制 $w \rightarrow w^T$

每 C 步: `q_target.load_state_dict(q.state_dict())`

$$L(w) = E[(r + \gamma \max_{a'} Q(s', a'; w^T) - Q(s, a; w))^2]$$

为什么它有效?

给回归一个稳定靶子

w^T 在 C 步内不变 $\Rightarrow y$ 在这段时间内是「固定标签」。监督学习的标准状况

梯度公式干净

$$\nabla_w L = E[(y - Q) \cdot (-\nabla_w Q(s, a; w))]$$

不需要追踪 y 内部的链式法则

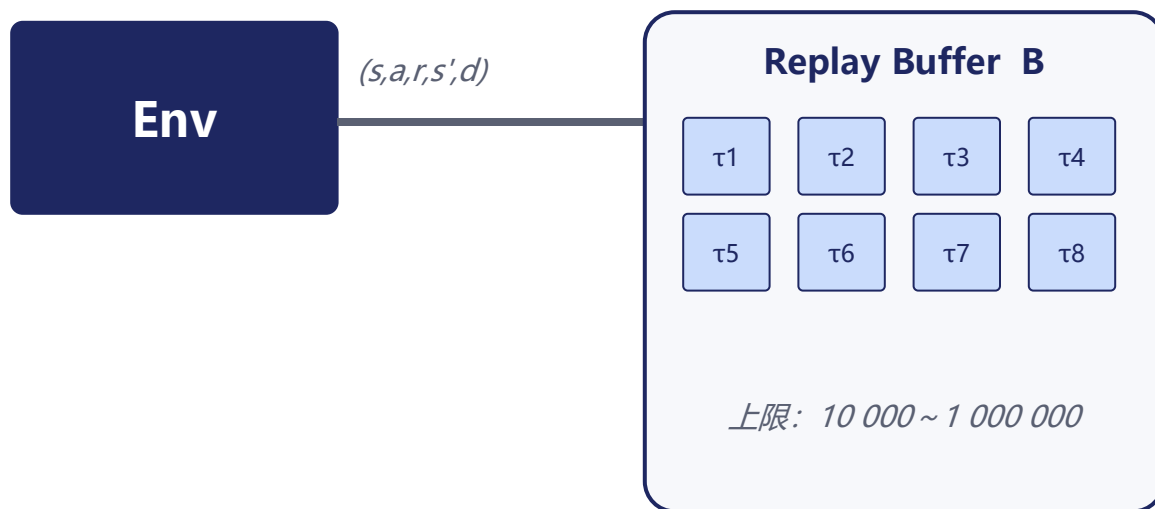
Lab09 参数

`SYNC_TARGET_FRAMES = 1000` $\Rightarrow$ 每 1000 步硬拷贝一次。

更平滑的写法是 Polyak 软更新: $w^T \leftarrow \tau w + (1 - \tau)w^T$

关键技术 ② — 经验回放 (Experience Replay)

把交互生成的 transition 存进一个大池子



训练时：均匀随机采小批量 $\Rightarrow$ 梯度更新

```
batch = rb.sample(BATCH_SIZE)
# 32-256 条 transition, 已经被打散
```

三大好处

① 打破相关性

同一回合里相邻样本高度相关，会让梯度有偏；均匀抽样近似 i.i.d.

② 重复利用样本

一条 transition 可被多次采样 $\Rightarrow$ 数据效率高，尤其在采样昂贵的环境。

③ 让 off-policy 合法

Q-learning 目标用贪心动作，与采样动作 (ϵ -greedy) 不一致。
回放池里的数据本来就是「旧策略」产生的 —— 完美契合 off-policy。

为什么回放必须「均匀」采样？

$$L(w) = E_{(S,A) \sim d} \left[\left(R + \gamma \max_{a'} Q(s', a'; w^T) - Q(S, A; w) \right)^2 \right]$$

R, S' 的分布

由环境模型 $p(R, S' | S, A)$ 决定 —— 物理事实，不可调。

(S, A) 的分布 d

需要我们选 —— DQN 没有「给定策略」，自然选「均匀分布」（无先验）。

现实采样

经验是按策略 π_b 序贯产生的，分布并不均匀 —— 所以要用 buffer + 均匀抽样矫正。

- 表格 Q-learning 不需要回放，因为它逐 (s, a) 更新，不依赖任何分布
- DQN 是「最小化标量损失 + 均匀样本」 = 一个非常标准的监督学习设定
- 进阶：Prioritized Experience Replay 用 TD-error 作权重 —— 是均匀的有偏改良版

DQN 算法完整伪代码

```
Initialize 主网络  $q(\cdot; w)$  和 目标网络  $q(\cdot; w^T)$ ,  $w^T \leftarrow w$ 
Initialize 回放缓冲区  $B = \emptyset$ 
for episode = 1, 2, ... do
     $s \leftarrow \text{env.reset}()$ 
    while not done:
         $a \leftarrow \epsilon\text{-greedy } w.r.t. \ q(s, \cdot; w)$            # 探索 vs 利用
         $s', r, d \leftarrow \text{env.step}(a)$                        # 与环境交互一步
         $B \leftarrow B \cup \{(s, a, r, s', d)\}$                  # 存入回放池
         $s \leftarrow s'$ 
        if  $|B| \geq \text{warmup}$ :
            batch = uniform_sample( $B$ ,  $N$ )                     # 均匀采小批量
             $y = r + \gamma (1 - d) \cdot \max_{a'} q(s', a'; w^T)$  # 目标网络算  $y$ 
             $L = (1/N) \sum (y - q(s, a; w))^2$                  # 平方 Bellman 误差
             $w \leftarrow w - \alpha \nabla_w L$                    # 反向传播
        if step %  $C == 0$ :
             $w^T \leftarrow w$                                    # 硬更新目标网络
```



第二部分 · 代码详解

Atari Pong 环境：状态 / 动作 / 奖励

状态 s

原始 RGB 帧

shape = (210, 160, 3)

uint8 像素，太大也太抖

→ 需要做预处理

动作 a

离散 6 个：

0 NOOP · 1 FIRE

2 RIGHT · 3 LEFT

4 UP · 5 DOWN

奖励 r

得分 = +1

失分 = -1

其余 0

DQN 训练时会被 clip 到 [-1, +1]
(已包含在转换链里)

↪ 代码入口 (Lab09)

```
base_env = GymEnv("ALE/Pong-v5",  
                  from_pixels=True,    # 用像素而非 RAM 作观测  
                  pixels_only=True,  
                  render_mode="rgb_array")  
n_actions = base_env.action_space.n    # = 6
```

图像预处理：把 $210 \times 160 \times 3 \rightarrow 4 \times 84 \times 84$

Why every step matters

1. FrameSkip $\times 4$

同一动作连续执行 4 帧，提速 $\sim 4\times$ ；
做 RL 时常和 max-pool 一起使用消除闪烁。

2. TimeMaxPool $T=2$

对最近 2 帧逐像素取 max，
抵消 Atari 偶数帧/奇数帧绘制差异。

3. ToTensorImage

HWC uint8 $\rightarrow$ CHW float $[0,1]$
归一化便于网络训练。

4. GrayScale

RGB $\rightarrow$ 灰度，通道数 $3 \rightarrow 1$
色彩对 Pong 无信息量。

5. Resize $84 \times 110 \rightarrow$ Crop 84×84

标准 DQN 输入尺寸，
裁掉记分板等无关像素。

6. CatFrames $N=4$

堆叠最近 4 帧 $\rightarrow (4, 84, 84)$
让网络感知小球的运动方向与速度。

TorchRL × TensorDict: 统一的数据容器

Why TensorDict, and what's in it

- ▶ **TensorDict** = dict + 共享 batch_size 的张量容器
- ▶ 环境每一步返回的就是一个 TensorDict
- ▶ `td.get("pixels")` → 当前观测
- ▶ `td.get(("next", "pixels"))` → 下一观测
- ▶ `td.get(("next", "reward"))` → 奖励
- ▶ 嵌套 next/* 是 RL 标准结构: 「这一步」与「下一步」清晰分离
- ▶ RB (ReplayBuffer) 原生支持 TensorDict, 无需手写打包逻辑
- ▶ **LazyMemmapStorage** 把样本写入磁盘 memmap, 省内存

```
td = TensorDict {  
    "pixels"                (4, 84, 84) float32  
    "action"                (6,) one-hot  
    ("next", "pixels")      (4, 84, 84)  
    ("next", "reward")      (1,) float  
    ("next", "done")        (1,) bool  
}
```

环境构建: TransformedEnv + 变换链

Lab09 env block, line by line

```
1 env = TransformedEnv(  
2     base_env,  
3     Compose(  
4         FrameSkipTransform(frame_skip=4),           # 跳帧加速  
5         TimeMaxPool(in_keys=['pixels'], T=2),        # 闪烁补偿  
6         ToTensorImage(from_int=True, in_keys=['pixels']), # uint8→float [0,1]  
7         GrayScale(in_keys=['pixels']),               # 3 通道 → 1 通道  
8         Resize(84, 110, in_keys=['pixels']),         # 先 resize  
9         CenterCrop(84, 84, in_keys=['pixels']),      # 再 crop  
10        CatFrames(N=4, dim=-3, in_keys=['pixels']),  # 堆叠 4 帧  
11        RewardClipping(-1, 1),                       # 奖励裁剪  
12    ),  
13 )
```

⚠ 变换的「顺序」很重要

ToTensorImage 必须在 *GrayScale* / *Resize* 之前; *CatFrames* 必须最后 — 否则张量维度对不上。

经验回放：用 TorchRL 写一行就够

Replay buffer setup

```
1 rb = TensorDictReplayBuffer(  
2     storage = LazyMemmapStorage(max_size=REPLAY_SIZE), # 磁盘 memmap, 10_000 条  
3     sampler = RandomSampler(), # 均匀随机采样  
4     batch_size = BATCH_SIZE, # 默认 64  
5 )
```

LazyMemmapStorage

数据写到磁盘内存映射文件，省内存
图像样本可以塞下百万条

RandomSampler

无放回 / 有放回均可
→ 满足均匀分布假设

batch_size

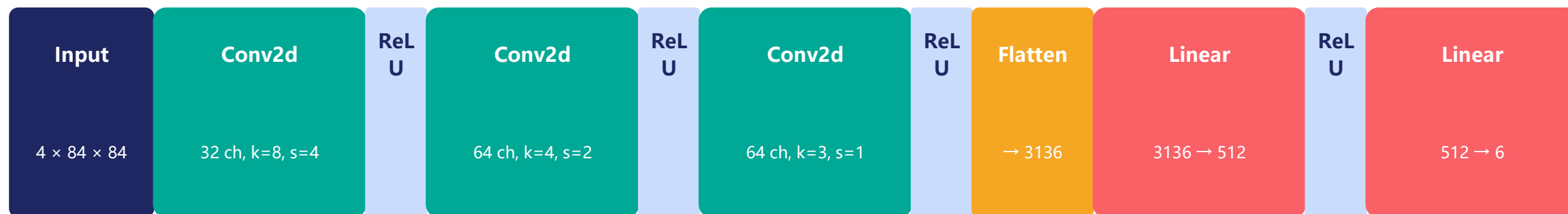
32 ~ 256 常见
Lab09 用 64
大 batch 训练更稳，但显存吃紧

✂ 插入一条 transition:

```
rb.add(TensorDict({'obs': obs, 'action': a, 'reward': r, 'next_obs': next_obs, 'done': d}, batch_size=[]))
```

Q 网络：经典 Nature-DQN CNN 结构

$3 \times \text{Conv} + 2 \times \text{FC}$, output dim = n_{actions}



 **维度计算 (公式: $\text{out} = \text{floor}((\text{in} - k) / s) + 1$)**

84 $\rightarrow$ Conv($k=8, s=4$): $(84-8)/4 + 1 = 20$

20 $\rightarrow$ Conv($k=4, s=2$): $(20-4)/2 + 1 = 9$

9 $\rightarrow$ Conv($k=3, s=1$): $(9-3)/1 + 1 = 7 \Rightarrow 64 \times 7 \times 7 = 3136$

输出层: $n_{\text{actions}} = 6$ 个 Q 值, 对应 Pong 的 6 个离散动作

Q 网络代码: class DQN(nn.Module)

Lab09 line 68 ~ 92

```
1 class DQN(nn.Module):
2     def __init__(self, n_actions):
3         super().__init__()
4         self.conv1 = nn.Conv2d(4, 32, kernel_size=8, stride=4)
5         self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)
6         self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)
7         self.flatten = nn.Flatten()
8         self.fc1conv = nn.Linear(64*7*7, 512)      # 3136 → 512
9         self.fc2conv = nn.Linear(512, n_actions)   # 512 → 6
10        self.relu = nn.ReLU()
11
12    def forward(self, x):
13        # x: (B, 4, 84, 84) float32 in [0,1]
14        x = self.relu(self.conv1(x))
15        x = self.relu(self.conv2(x))
16        x = self.relu(self.conv3(x))
17        x = self.flatten(x)                # (B, 3136)
18        x = self.relu(self.fc1conv(x))     # (B, 512)
19        return self.fc2conv(x)            # (B, 6) ← 6 个 Q 值
```

几个关键点

4 → 32 → 64 → 64 通道

通道随网络加深; 空间分辨率随 stride 变小

无 BatchNorm

DQN 极少用 BN; Layer/GroupNorm 也少见

ReLU 共享

self.relu 是无状态层, 复用一份实例即可

没有 Dropout

RL 训练已经够噪了, 再 Dropout 反而不稳

主网络 + 目标网络的初始化

Two networks, same architecture

```
q = DQN(n_actions).to(device)
q_target = DQN(n_actions).to(device)
q_target.load_state_dict(q.state_dict()) # 关键: 参数初始时完全一致
q_target.eval()                        # 关掉 Dropout/BN (即使没用到, 养成习惯)

optimizer = optim.Adam(q.parameters(), lr=LEARNING_RATE)
```

load_state_dict

把 q 的全部权重拷贝给 q_target —— 而不是把 q_target 引用到 q。
二者从此独立，互不影响。

eval() 模式

BN/Dropout 在 eval 下表现固定；DQN 这里其实没启用它们，
但仍是一个良好实践。

只优化 q

optimizer 只接收 q.parameters() —— q_target 不学，靠硬同步更新；
永远不要把 q_target.parameters() 也加进去。

ε-greedy 策略：探索 vs 利用

How DQN picks an action

ε 线性退火

$$\varepsilon = \max(\varepsilon_{\min}, \varepsilon_{\text{start}} - \text{frame} / \text{decay})$$

- ▶ EPSILON_START = 1.00
- ▶ EPSILON_FINAL = 0.01
- ▶ EPSILON_DECAY_LAST = 150 000 步
- ▶ 初期几乎纯随机 → 后期几乎全贪心
- ▶ 退火过快 ⇒ 探索不够；过慢 ⇒ 浪费样本

```
if torch.rand(1).item() < epsilon:
    a = env.action_spec.rand()          # 随机动作
else:
    # 贪心
    with torch.no_grad():
        x = obs.unsqueeze(0).to(device) # (1,4,84,84)
        qvals = q(x)                    # (1,6)
        idx = torch.argmax(qvals, dim=1)
        a = torch.zeros(6); a[idx] = 1.0 # one-hot
```

 **注意：a 是 one-hot 而非整数**

TorchRL 的 action spec 期望 one-hot, 环境也用 one-hot 推进；后面 gather 时再用 argmax 转回索引。

训练循环 ①：与环境交互、存样本

Loop body, environment-interaction half

```
while True:
    frame_idx += 1
    epsilon = max(EPSILON_FINAL, EPSILON_START - frame_idx / EPSILON_DECAY_LAST_FRAME)
    obs = next_obs
    a = epsilon_greedy(obs, epsilon, q, env)

    td = env.step(td.set('action', a))      # 推进一步
    next_obs = td.get(('next', 'pixels')).float()
    r = td.get(('next', 'reward'))
    d = td.get(('next', 'done'))

    transition = TensorDict(
        {'obs': obs, 'action': a, 'reward': r,
         'next_obs': next_obs, 'done': d}, batch_size=[])
    rb.add(transition)                      # 写入回放池

    if d.item():                          # 一局结束
        td = env.reset(); next_obs = td.get('pixels').float()
```


训练循环 ②：从回放池更新主网络

Loop body, learning half (Double DQN)

```
if len(rb) >= REPLAY_START_SIZE:                # 先等池子有 10_000 条
    batch = rb.sample(BATCH_SIZE)                # 均匀采 64 条
    obs_b = batch['obs'].to(device)
    act_b = batch['action'].long().to(device)
    rew_b = batch['reward'].to(device).squeeze(-1)
    next_obs_b = batch['next_obs'].to(device)
    done_b = batch['done'].to(device).float().squeeze(-1)

    with torch.no_grad():
        # ★ Double DQN: 动作由主网络挑, 估值由目标网络给
        opt_act = q(next_obs_b).argmax(1)
        q_next = q_target(next_obs_b).gather(1, opt_act.unsqueeze(-1)).squeeze(1)
        target = rew_b + gamma * (1.0 - done_b) * q_next

    act_b_ind = act_b.argmax(dim=-1)              # one-hot → 索引
    q_values = q(obs_b).gather(1, act_b_ind.unsqueeze(-1)).squeeze(1)
    loss = F.mse_loss(q_values, target)

    optimizer.zero_grad(); loss.backward(); optimizer.step()
    if frame_idx % SYNC_TARGET_FRAMES == 0:
        q_target.load_state_dict(q.state_dict()) # 硬同步
```

代码亮点: Double DQN (不是普通 DQN)

Decoupling action-selection from evaluation

原始 DQN (Nature, 2015)

$$y = r + \gamma \cdot \max_{a'} Q(s', a'; w^T)$$

问题

- ▶ max 操作同时做「挑动作」和「估值」
- ▶ 两者都来自同一个有噪估计 → 系统性 高估 (overestimation)
- ▶ 训练后期价值膨胀, 策略反而退化

Double DQN (van Hasselt, 2016)

$$y = r + \gamma \cdot Q(s', \arg\max_{a'} Q(s', a'; w); w^T)$$

改进

- ▶ **主网络 q** 负责挑动作 $a^* = \arg\max$
- ▶ **目标网络 q^T** 负责给 $Q(s', a^*)$
- ▶ 两次估计的噪声不再相关 → 显著减弱高估
- ▶ Lab09 用的就是这种写法

代码细节: gather 是怎么取 $Q(s, a)$ 的?

torch.gather, dim=1, index=action_idx

网络输出 (B=4, n_actions=6)

+0.1	-0.2	+0.4	+0.0	+0.3	-0.1
-0.3	+0.5	+0.1	+0.2	+0.0	+0.4
+0.2	+0.1	-0.1	+0.6	+0.3	-0.2
+0.4	+0.0	+0.2	+0.1	-0.3	+0.5

act_b_ind = [2, 1, 3, 5] → gather 沿 dim=1 拿这些列

```
q_values = q(obs_b)\
            .gather(1, act_b_ind.unsqueeze(-1))\
            .squeeze(1)
```

结果 shape: (B,) 即每个样本一个 $Q(s, a)$

为什么要 unsqueeze?

gather 要求 index 与输入维度相同: $(B, 6) \leftrightarrow (B, 1)$ 。
unsqueeze 把 $(B,)$ 变 $(B, 1)$ 。

为什么要 squeeze?

gather 输出仍是 $(B, 1)$; squeeze(1) 把多余的列维去掉, 方便和 target $(B,)$ 求 MSE。

损失函数选择：MSE vs Huber (Smooth L1)

F.mse_loss — Lab08 用的

$$L = (Q - y)^2$$

- ▶ 数学最干净，梯度恒为 $2(Q - y)$
- ▶ 大误差 → 大梯度 → 容易爆炸
- ▶ DQN 早期 / Pong 这种 reward clip 到 $[-1, 1]$ 的环境一般够用
- ▶ 需要配合 Adam 或者梯度裁剪

F.smooth_l1_loss — 推荐

$$L = \frac{1}{2}(Q - y)^2 \text{ if } |\cdot| < 1$$
$$L = |Q - y| - \frac{1}{2} \text{ 否则}$$

- ▶ 小误差像 MSE（平滑），大误差像 L1（线性）
- ▶ 对离群梯度更鲁棒
- ▶ 原始 DeepMind 2015 实现用的就是 Huber
- ▶ 把 Lab08 的 mse_loss 换成 smooth_l1_loss 一般会更稳

Lab09 的关键超参数

Hyper-params at a glance

参数	值	含义
gamma	0.99	折扣，决定看多远
BATCH_SIZE	64	小批量采样大小
REPLAY_SIZE	10 000	回放池上限 (Pong 实际建议 100k+)
REPLAY_START_SIZE	10 000	训练前的热身样本
LEARNING_RATE	1e-4	Adam 学习率
SYNC_TARGET_FRAMES	1 000	每 1000 帧硬同步一次目标网
EPSILON_START	1.00	起始 ϵ (纯随机)
EPSILON_FINAL	0.01	末端 ϵ (接近贪心)
EPSILON_DECAY_LAST	150 000	ϵ 线性下降到 final 用了 15w 帧

常见 Bug & 编程陷阱

Things students get wrong all the time

⚠ batch_size 未定义

Lab08 的 Hint 2 直接用了 batch_size 但没赋值。训练循环要先 `batch = rb.sample(BATCH_SIZE)`。

⚠ one-hot ↔ 索引 没转换

action 是 one-hot 时，必须先 `act.argmax(dim=-1)` 再 `gather`；否则 index 越界。

⚠ 忘记 `torch.no_grad()` 包目标计算

目标 `y` 不该参与反向传播，否则 `q_target` 也会被更新 ⇒ 全部失稳。

⚠ done 没 cast 成 float

`(1 - done_b)` 用 `bool` 会报类型错；要 `done_b.float()` 或 `squeeze(-1).float()`。

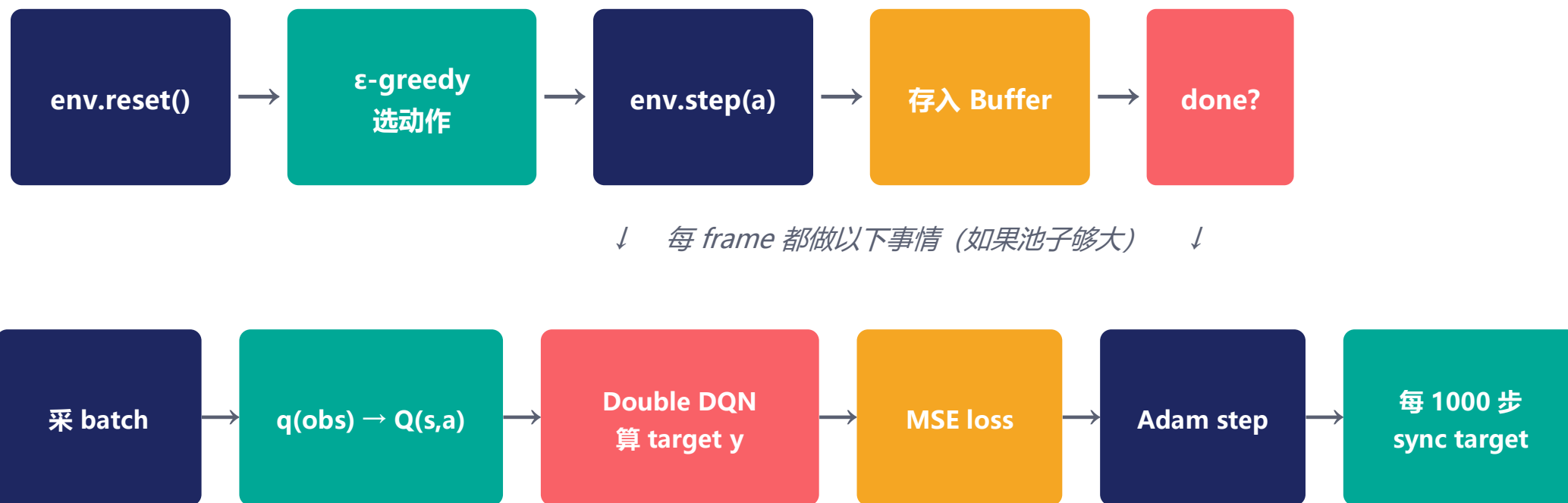
⚠ target 网络忘记同步

`frame_idx % SYNC_TARGET_FRAMES == 0` 这一行注释掉过 ⇒ 永远学不出策略。

⚠ 池子还没满就开训

需要 `if len(rb) >= REPLAY_START_SIZE`，否则采样分布过窄，早期方差爆炸。

完整训练流程：一图回顾



一句话总结

DQN = 用「神经网络」近似 Q^* , 用「均匀经验回放」消除样本相关, 用「目标网络」给一个稳定的回归靶子; 剩下的就是把 MSE / Huber 误差降到底。